



Revision Notes: Sorting Algorithms - Count Sort & Merge Sort

Introduction

In this session, we focused on two primary sorting algorithms: Count Sort and Merge Sort. These algorithms offer different strategies for sorting arrays and have distinct characteristics with regard to time complexity, stability, and whether they are in-place sorts.

Concepts Covered

1. Smallest Number from Single Digits

- **Problem Statement:** We explored a problem where we rearrange digits of a given number to find the smallest possible number. This served as an introduction to understanding the Count Sort algorithm.
- **Example:** Given an array `[6, 3, 4, 2, 7, 2, 1]`, we derive the smallest number `[1, 2, 2, 3, 4, 6, 7]`.
- **Approach:** Use a frequency array of size 10 (representing digits 0-9) to count occurrences and then reconstruct the number `["6:3+source"]`.

2. Count Sort

- **Definition:** A sorting algorithm that uses the frequency of elements to sort a collection with a known range.
- **Use Case:** Best for sorting numbers with a fixed range (commonly 0-9 for digits), as it achieves $O(N)$ time complexity. This is efficient compared to traditional $O(N \log N)$ algorithms for small ranges.
- **Algorithm Steps:**
 1. Create a frequency array to count occurrences of each digit.
 2. Reconstruct the sorted array using the frequency data.
 3. Given constraints (e.g., large numbers), Count Sort is practical for arrays up to the range of 10^6 .



smallest number .

3. Merge Sort

- **Definition:** A divide-and-conquer algorithm that divides an array into two halves, recursively sorts them, and then merges the sorted halves.
- **Characteristics:**
 - **Time Complexity:** $O(N \log N)$, as each level of the recursion tree requires $O(N)$ time for merging.
 - **Space Complexity:** $O(N)$, due to the auxiliary space for merging and recursion stack .
 - **Stability:** Merge Sort is stable as it maintains the relative order of similar elements by sorting based on one parameter at a time.
- **Algorithm Steps:**
 1. **Divide:** Split the array into two halves.
 2. **Conquer:** Recursively sort each half.
 3. **Combine:** Merge the sorted halves to produce the sorted array .

4. Stable and In-Place Sorting

- **Stable Sort:** Maintains the relative order of records with equal keys (e.g., maintaining the order of identical elements as they appear in the input).
- **In-Place Sort:** Sorting where the algorithm handles the sorting by modifying the original data structure without requiring extra space. The space complexity is ideally $O(1)$.

Real-World Application and Analogy

An analogy used in the class was comparing stable sorts to stable marriages, where each partner keeps their original relative order . Count Sort, although efficient, is analogous to counting items for quick sorting — ideal for a limited range, similar to sorting colored balls into bins by color.

Conclusion

These sorting techniques offer different advantages. Count Sort is particularly effective for small ranges due to its linearity, whereas



for tackling various sorting challenges efficiently.